

Asset-Defender Project Questions & Answers

Consolidated Q&A (Your 23 + Additional 20)

Generated for project: D:\instagram final\Asset-Defender

1. What is the workflow of my project?

User logs in -> extension scrapes profile/messages on Instagram -> client heuristics compute preliminary signals -> backend runs deep scoring + ML inference -> risk/classification is stored in SQLite -> results shown in popup/dashboard/admin view.

2. What is my literature survey and where will I find it?

A dedicated literature-survey document is not present in the repository. Closest references are training scripts, model UI cards, and requirement notes.

3. What is my methodology?

Hybrid detection: (1) deterministic heuristics, (2) ML model predictions, (3) confidence/evidence gating, (4) final fusion and rule overrides (especially message phishing cues).

4. How many modules have been used?

Major functional modules: auth, extension messaging, scraping, client scoring, server analysis API, ML inference/training, storage, admin analytics (about 8 major modules).

5. How many machine learning models are used and why?

6 classifiers + 1 vectorizer:

Profile: Random Forest, XGBoost, Gradient Boosting, Logistic Regression.

Message: Multinomial Naive Bayes, Logistic Regression.

NLP preprocessing: TF-IDF vectorizer.

Runtime uses primary/backup for reliability and failover.

6. How do you predict heuristic scores, preliminary risks, and UI output?

In popup.js, profile and message heuristics compute structural/content/behavior/photo and message-risk dimensions. Weighted trust is converted to preliminary risk, confidence, and explanation reasons displayed in popup UI.

7. How do you calculate the final risk score?

Current extension fusion: $\text{finalRisk} = 0.3 * \text{profileRisk} + 0.7 * \text{messageRisk}$ (when both exist).
Message phishing/scam overrides can force scam/hacker label even with strong profile cues.

8. How do you calculate client-side message analysis?

Client computes spamScore, urgentScore, burstScore from keywords, suspicious links/domains, repetition, burst timing, caps/emoji signals, then aggregates to overallRisk with a weighted formula and scam-floor logic.

9. How do you calculate server-side message analysis?

Server extracts lexical, credential, impersonation, financial pressure, domain, repetition, and timing features; computes threatScore; applies floor rules; merges with ML message risk by taking max(heuristic, ML) for effective risk.

10. What rules are used for client and server calculations?

Client: trust dimensions, behavioral evidence gates, ratio/institutional rules, explicit scam signal penalties.

Server: weighted threat formula, classification tags, calibration for profile risk, confidence scoring, and insufficient-data logic.

11. What are the file names for client-side profile/message prediction?

Main client prediction logic: client/public/popup.js.

Scraping/source extraction: client/public/instagram_analyzer.js.

12. What is the file name for database storing?

Database schema and writes: server/database.js.

Main DB file: server/database/app.db.

13. How many databases are used?

Primary app DB: 1 SQLite (app.db).

Also a secondary ML-side SQLite path exists in server/ml/database.py (asset_defender.db), mainly for Flask ML script path.

14. Is the project usable for multiple users?

Yes. JWT includes userId, analyses are stored with user_id, and admin endpoints can monitor all users.

15. What formulas are used for client analysis, server analysis, and final prediction?

Client final fusion: $0.3 \times \text{profile} + 0.7 \times \text{message}$.

Client profile preliminaryRisk: $(100 - \text{finalTrust}) + \text{uncertaintyPenalty} + \text{negativeSignalPenalty}$ (clamped).

Client message risk: weighted urgent/behavioral/spam.

Server effective risk: $\max(\text{mlRisk}, \text{heuristicRisk})$.

Server profile calibration: sigmoid + data-quality blending.

16. How is Instagram data scraped?

Content scripts parse DOM and embedded JSON/API responses on instagram.com, scrape profile stats/media/message text/timing, and use background service worker tab orchestration for DM/profile contexts.

17. Is scraping violating government rules?

No legal determination is encoded in software. Compliance depends on Instagram ToS, privacy law, user consent, and jurisdiction. Legal review is required before production deployment.

18. How are you connecting to the extension layer?

Manifest V3 architecture: popup + background service worker + content scripts, connected via `chrome.runtime.sendMessage/chrome.tabs.sendMessage` and `chrome.storage.local`.

19. How do you maintain the extension layer?

Pending-task persistence in storage, background tab lifecycle management, auth session sync, async result forwarding from hidden analysis tabs to popup.

20. How many ML models, NLP models, AI models are used?

6 ML classifiers and 1 TF-IDF NLP vectorizer are present in the current implementation.

21. How can admin control all users' data?

Admin JWT + role check protects `/api/admin/users`, `/api/admin/analyses`, `/api/admin/stats`. Admin dashboard supports viewing/filtering and CSV export.

22. What datasets are used for training ML models?

Profile model dataset: `profiledataset/train.csv`.

Message model dataset: `messagedatasets/spam.csv`.

23. How are heuristic scores, sentiment, and behavioral analysis calculated?

Heuristic/rule formulas are used for scoring. Behavioral metrics include engagement variability, posting intervals, repetition/burst patterns, and interaction density. Sentiment is proxy-based (toxic/promotional/risk terms), not a dedicated transformer sentiment model.

24. What are exact limitations/failure cases (private profiles, blocked scraping, low sample, language mismatch)?

Limitations include private/blocked pages, DOM changes, missing deep results, insufficient media/message samples, and weaker support for non-English or obfuscated text.

25. What is the ground-truth labeling process for training data quality?

No in-repo formal labeling workflow is implemented. Ground truth currently comes from dataset labels in `train.csv` and `spam.csv`.

26. What are model evaluation metrics beyond accuracy (precision, recall, F1, ROC-AUC, confusion matrix)?

Current training scripts primarily output accuracy. Additional metrics are not yet reported in the current pipeline.

27. How do you handle false positives vs false negatives tradeoff?

Through threshold gates, insufficient-data suppression, and message-dominant phishing overrides. There is no explicit cost-sensitive optimization framework yet.

28. What is threshold tuning strategy and why cutoffs?

Cutoffs are rule-driven/manual and aligned to practical risk bands (e.g., medium/high). No automated threshold optimization notebook/pipeline is included.

29. How do you handle model drift and retraining schedule?

No periodic retraining scheduler is implemented. Retraining is currently manual or triggered when model files are absent.

30. What is versioning strategy for models/rules (rollback support)?

Primary/backup model failover exists. Full model registry/version tags and rollback workflow are not yet implemented.

31. What are security controls (JWT expiry, secret rotation, password policy, rate limiting)?

Implemented: bcrypt hashing, JWT auth with expiry, role-based admin authorization, CORS.
Missing/needs hardening: secret rotation policy, strict password policy, API rate limiting, CSRF/security headers/audit controls.

32. What is privacy/data retention policy (stored data, duration, deletion path)?

Stored data includes user account details and analysis records in SQLite. No explicit retention/deletion policy automation is currently implemented in code.

33. What is legal/compliance scope (ToS, consent, jurisdiction boundaries)?

No formal legal policy document is present in the repository. Compliance scope should be defined before production usage.

34. How do you test end-to-end (unit/integration/E2E) and what coverage?

No dedicated first-party test suite/coverage report is found for the app logic in current repo state.

35. What is performance profile (latency per analysis, max concurrent users)?

No formal benchmark report is present. Latency/concurrency limits must be profiled under load tests.

36. How is error handling and observability done (logs, alerts, monitoring)?

Try/catch and console logging are present; there is also a Winston logger module in server/src/logger.ts. Alerting/monitoring stack is not fully wired in the core path.

37. What is deployment architecture (dev/staging/prod, env vars, CI/CD)?

Current configuration is development-centric (localhost endpoints, local SQLite, extension host permissions). No complete CI/CD or staged deployment setup is visible.

38. What explainability output is shown to users?

UI shows score breakdowns, risk class, confidence, evidence quality, and reason strings describing the key triggers for profile/message/final decision.

39. What is human-in-the-loop review flow for disputed predictions?

Admin can review analyses, but there is no dedicated dispute queue, adjudication lifecycle, or feedback loop that retrains models from reviewed cases.

40. What are abuse/adversarial scenarios (prompting evasion, obfuscated links)?

Partially covered via suspicious-domain/TLD and keyword/repetition features. Advanced adversarial handling (unicode/homoglyph obfuscation, multilingual evasion) is limited.

41. What is backup/recovery plan for DB and model artifacts?

No explicit automated backup/recovery scripts are present for SQLite DB and model artifacts.

42. How do you ensure multi-tenant isolation between users' data?

Logical isolation via JWT userId and user_id-scoped queries. Physical tenant isolation is not present (shared SQLite).

43. What is roadmap (next model improvements, multilingual support, API hardening)?

Recommended roadmap: multilingual risk models, evaluation metric expansion, threshold tuning pipeline, model version registry, rate limiting/security headers, test automation, CI/CD, and legal/privacy policy formalization.